🪜

GODSHAND Test Ladder — Fast Iteration on Free Kaggle

> ⚡ **The rule:** climb only when the current rung is green. Most questions die (or pass) on Rung 0–1 — seconds to ~2 min — so you almost never pay the 20-min render tax to answer something a log line could tell you.

The cost model you're fighting: **total wall-clock per *validated change* = (setup + wait + render) × iterations.** Free Kaggle inflates all three. This ladder shrinks each iteration; the infra fixes shrink the setup tax.

The ladder

Rung 0 — Sandbox, no GPU, seconds

Does the code even read my change? Run before opening Kaggle.

- Confirm the env var is actually consumed by the code path:
  `rg "KIT_FLF2V_ENDFRAME|KIT_MOTION_DENOISE|KIT_FACE_ENHANCE" src/`
- Confirm the config parses and the active profile is what you expect:
  `python3 -c "import yaml;d=yaml.safe_load(open('config_confrontation.yaml'));print(d['active_profile'])"`
- **Catches:** typo'd var names, editing a var the code doesn't read, wrong profile active. ~90% of "why didn't my change do anything" bugs die here, for free.

Rung 1 — Kaggle session, graph-wiring only, ~1–2 min

Did my change reach the graph? Let the graph build, read the `[graph]` lines, then kill before sampling.

- Look for the echo: `[graph] ... 4 steps split@2 | cfg 1.0 | 768x512 len 41`, `[graph] spatial upscaler NOT wired`, the denoise value, and whether the **FLF2V/end-frame path** was chosen.
- **Catches:** end-frame morph silently not engaging, upscaler still unwired, denoise not applied. Answers most current open questions with zero real frames rendered.

Rung 2 — Single shot, smoke mode, ~2–4 min

Does the mechanism actually move? `--smoke` (caps steps at 12) + one shot (`GODSHAND_PROOF_SHOTS=1`, e.g. `i01_jungle`).

- Not final quality — checking *is motion frozen or moving, did grade run, did morph fire.* Testing the variable, not a deliverable.
- **Catches:** frozen motion, dead grade, broken morph — at ~1/8th the cost of a real shot.

Rung 3 — Single shot, full quality, ~20 min

Is quality good on ONE shot? Full steps + grade + post on one representative shot. Only climb here once 0–2 are green. Now judge sharpness, grade, motion fidelity for real.

Rung 4 — Proof set, 4 shots, ~80 min

Is it consistent across shots? Current proof mode — cross-shot coherence + the actual proof deliverable.

Rung 5 — Full 28, `GODSHAND_MODE=full`, 10–12 GPU-hr

Production. Only after the proof passes the quality gate.

Infra fixes that lower the tax on every rung

- **Model cache as a Kaggle Dataset** (`kkk3213/godshand`) attached as input → models stop re-downloading each session. Biggest single cut to the cold-start tax.
- **Resume, don't re-render:** `KIT_FORCE_RERENDER=0` + add previous Output as Input → finished clips aren't recomputed.

- **Batch hypotheses per session:** use Rungs 0–1 to pre-filter which changes are worth a render, then test several survivors in one GPU session.

Mapped to live open questions

Question	Answered on	Cost
End-frame morph engaged?	Rung 0 + Rung 1	seconds
Is motion frozen?	Rung 2 (smoke single shot)	~3 min
Is the grade running?	Rung 1 (`[grade] ->` line)	seconds
Sharpness / upscaler wired?	Rung 1 (`spatial upscaler NOT wired`)	seconds

Ready-to-paste Kaggle cell

📝 Adjust `KIT_DIR` and `LOG` to your Kaggle paths. Point `LOG` at wherever you capture cell output (e.g. `tee` a run into a file). Edit the knob block to the change you're testing, then run one rung at a time.

```
# ====================== GODSHAND TEST LADDER ================
=====
KIT_DIR=/kaggle/working/GODSHAND_FINAL/kit   # <-- adjust
LOG=/kaggle/working/run.log                  # <-- where you
tee output
cd "$KIT_DIR"

# ---- knobs under test (edit these) ----
export KIT_CONFIG=config_confrontation.yaml
export KIT_RENDER_MODE=motion_stills
export KIT_FLF2V_ENDFRAME=1
export KIT_MOTION_DENOISE=0.60
export KIT_FACE_ENHANCE=1
export KIT_FAST_POST=0
export KIT_FORCE_RERENDER=0     # resume, don't recompute fin
ished clips
```

```
export CONFIRM_FULL_RUN=1

# ---- Rung 0: no GPU, seconds (code + config sanity) ----
rg "KIT_FLF2V_ENDFRAME|KIT_MOTION_DENOISE|KIT_FACE_ENHANCE" s
rc/
python3 -c "import yaml;d=yaml.safe_load(open('config_confron
tation.yaml'));print('active_profile:',d['active_profile'])"

# ---- Rung 1: graph-wiring only (~1-2 min) — kill after [gra
ph] echoes ----
# timeout 120 python run.py --config "$KIT_CONFIG" 2>&1 | tee
"$LOG"

# ---- Rung 2: single shot, smoke (~2-4 min) ----
# export GODSHAND_MODE=proof GODSHAND_PROOF_SHOTS=1
# python run.py --config "$KIT_CONFIG" --smoke 2>&1 | tee "$L
OG"

# ---- Rung 3: single shot, full quality (~20 min) ----
# export GODSHAND_MODE=proof GODSHAND_PROOF_SHOTS=1
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- Rung 4: proof set, 4 shots (~80 min) ----
# export GODSHAND_MODE=proof
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- Rung 5: full 28 (10-12 GPU-hr) ----
# export GODSHAND_MODE=full KIT_MAX_HOURS=11
# python run.py --config "$KIT_CONFIG" 2>&1 | tee "$LOG"

# ---- read the answer in one line (any rung) ----
rg "\[graph\]|\[preflight\]|\[grade\]|\[merge\]|\[resume\]"
"$LOG" | tail -30
```

⚠️ `run.py` flags/entry may differ from your kit — reconcile against `MASTER.sh` / `run.py --help`. The `--smoke` cap (12 steps) and env-var names are verified from the kit source; the exact CLI invocation is the one thing to confirm on first use.